



Bibliography

P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 15, 2026

Contents

1	Problem Context	2
1.1	Scenario and Pain Points	2
1.2	Problem Definition	2
2	Overview of Existing Work	3
3	Comparison of Approaches	3
4	Identified Gap	4
4.1	Limitations of Centralized Solutions	4
4.2	Limitations of Existing Peer-to-Peer Solutions	4
4.3	The Gap	4
5	Positioning Your Project	5
5.1	Key Differentiators	5
5.2	Design Philosophy	5
6	Research to Application Mapping	5
7	Key References	6
8	Conclusion	7

1 Problem Context

1.1 Scenario and Pain Points

To motivate the problem, we begin with a concrete collaborative scenario commonly observed in university projects.

University students frequently work in temporary groups and need to exchange files efficiently without relying on complex infrastructure, cloud account management, or external storage devices. During the course of a project, team members collaboratively produce and modify heterogeneous files, including source code, documents, images, datasets, and presentation materials. Since collaborators often use different operating systems and development environments, cross-platform compatibility further increases the complexity of file sharing and coordination.

In practice, temporary student teams usually lack the time, resources, and technical expertise required to maintain a dedicated file management or synchronization system. As a result, files are often exchanged through ad hoc communication channels such as group chats or email attachments. This approach introduces several issues, including duplicated files, inconsistent versions, lack of synchronization, and absence of proper change tracking.

In addition, ownership and persistence of project data become problematic once the collaboration ends. Some members may wish to preserve the project files for future reference, while others may prefer to discard them. However, because the files are typically distributed informally and managed inconsistently, long-term access often depends on a single individual maintaining the archive. This creates risks of data loss, incomplete project history, and fragmented ownership.

Therefore, the students require a lightweight and decentralized solution that enables them to:

- create a shared repository without complicated server deployment or administrative setup;
- support synchronization of heterogeneous file types across multiple platforms;
- track modifications and maintain version history;
- provide shared ownership of project data among collaborators; and
- avoid dependence on a single participant for long-term storage, archival, or deletion of the project files.

1.2 Problem Definition

Based on the scenario described above, small-scale collaborative groups require a file-sharing mechanism that is lightweight, decentralized, and easy to use. In such environments, participants mainly care about whether they can quickly form a shared workspace, exchange project files reliably, preserve previous versions when changes are made, and continue working with the shared folder as if it were a normal local directory.

From this objective, the problem can be defined as the design of a peer-to-peer directory-level synchronization system with lightweight version control. More specifically, the system must solve the following technical problems:

- **Local peer discovery and dynamic membership.** The system must discover available peers on the same local network without relying on a central server. It should also support dynamic participation, where peers may join, leave, or temporarily disconnect during collaboration.

- **Directory-level change detection and propagation.** The system must monitor a shared local directory and detect file creation, modification, and deletion events. These changes should be propagated to other online peers without requiring users to manually upload, download, or explicitly send files.
- **Rapid synchronization among online peers.** When a change occurs, the system should synchronize the updated state to other available peers within a short delay, so that collaborators can observe a consistent project workspace during active collaboration.
- **Support for heterogeneous file types and large files.** The system must handle different categories of project files, including text files, source code, documents, images, and binary files. For large files, the system should support efficient transfer mechanisms rather than assuming that every file can be sent as a single small message.
- **Simple versioning and rollback.** Before a file is overwritten by a synchronized update, the system should preserve an older copy or historical state. This provides a basic recovery mechanism against accidental overwrites, incorrect updates, or unintended file changes.
- **Transparent integration with the local file system.** The shared workspace should behave like a normal directory on the user's computer. Users should be able to create, edit, move, and delete files using their usual tools, while synchronization and version management are handled by the system in the background.

Therefore, the central technical problem is to provide a serverless, peer-to-peer, directory-level synchronization mechanism that supports dynamic local-network collaboration, heterogeneous file transfer, rapid update propagation, and basic version recovery while preserving the simplicity of a normal local folder.

2 Overview of Existing Work

We will go over the existing solutions...here. For application level: We need to check popular application settled in this scenario.... Code sharing: Github, gitlab...

File sharing: Google docs/google driver, Drop Box, Nextcloud

Directory and ownership sharing: Teams

P2p sharing: resilio sync, syncthing

P2p storage: IPFS, Tahoe-LAFS

For Technical level... we need to go through the existing distributed system and p2p...

Versioning method: Git, SVN, Perforce..

P2p protocols...

Sharing: Local Lan, over bluetooth, air drop, ...

3 Comparison of Approaches

This will be done based on what we have found in the previous on. Basically, we need to focus on:

Application Level:

Central server v.s. distributed

Directory sharing/file sharing

Large file support/binary file support

4 Identified Gap

University students and small collaborative teams face significant constraints when choosing file-sharing solutions for project work. The current landscape presents a fundamental gap: either users rely on centralized services with inherent limitations, or they resort to ad hoc peer-to-peer tools that lack critical versioning and conflict-resolution features.

4.1 Limitations of Centralized Solutions

Cloud-based file sharing services (Google Drive, GitHub, Microsoft Teams) introduce several technical and operational challenges:

- **Centralized infrastructure dependency:** These services require persistent internet connectivity and rely on third-party servers, introducing latency, availability risks, and potential data privacy concerns in institutional or air-gapped environments.
- **Limited file type and size support:** Platforms optimized for documents or source code often struggle with heterogeneous file types (datasets, binaries, media files) and impose storage quotas.
- **Communication friction:** Most file sharing occurs through secondary communication channels (email, messaging apps), separating file distribution from the actual collaboration workspace.
- **Ownership and permission bottlenecks:** Centralized ownership models (e.g., GitHub branch protection, Google Drive permissions) introduce approval delays and reduce team autonomy. Changes may require owner confirmation (e.g., merge requests), slowing iteration cycles during active sprints.

4.2 Limitations of Existing Peer-to-Peer Solutions

While peer-to-peer services (AirDrop, Syncthing) avoid centralization, they present their own constraints:

- **Absence of versioning:** Most P2P file-sharing tools provide no built-in history or rollback capability, making accidental overwrites irreversible.
- **Directory structure inconsistency:** Local P2P services often fail to maintain directory-level consistency and do not integrate seamlessly with the file system abstraction.
- **Limited conflict resolution:** Concurrent edits and synchronization failures lack deterministic handling strategies.

4.3 The Gap

The gap is clear: no lightweight, decentralized, directory-level file synchronization tool exists that combines the freedom of peer-to-peer sharing with the safety of version history, the simplicity of local network operation, and support for heterogeneous file types without permission bottlenecks.

5 Positioning Your Project

Our project addresses this gap by providing a lightweight, decentralized, directory-level file synchronization system designed specifically for local-network collaboration among small teams. Unlike centralized cloud services or generic P2P tools, our solution combines the best aspects of both approaches while remaining simple and portable.

5.1 Key Differentiators

- **Local network operation:** Our system operates entirely within the local network (WLAN/Bluetooth), eliminating dependency on external servers and providing immediate, latency-free collaboration.
- **Heterogeneous file support:** The system places no artificial restrictions on file types or sizes. Any file readable by the operating system can be synchronized. The only practical limit is the available disk space and the OS's own capabilities.
- **Built-in versioning and rollback:** Unlike most P2P solutions, our system automatically preserves previous versions of files before overwriting them, providing a safety net against accidental modifications or conflicts.
- **Lightweight and portable:** Designed as a simple, portable application with an intuitive user interface. No server configuration, administrative privileges, or complex setup required. Users simply select a folder and begin collaborating.
- **Transparent file system integration:** The shared directory behaves like a normal folder on the user's computer. Synchronization, versioning, and conflict resolution happen transparently in the background, without requiring users to master specialized workflows (like Git commits or merge requests).

5.2 Design Philosophy

Our approach prioritizes **simplicity**, **autonomy**, and **reliability**:

- **Simplicity:** Minimize user friction. File sharing and versioning should "just work" without requiring users to understand distributed systems, conflict resolution, or version control concepts.
- **Autonomy:** No ownership bottlenecks. Any peer can modify files; changes propagate to all peers without requiring approval or synchronization ceremonies.
- **Reliability:** Preserve data safety. Version history protects against accidental overwrites; deterministic conflict resolution ensures consistent behavior across all peers.

6 Research to Application Mapping

- **Zero-configuration peer discovery on LAN**
 - **Research Insight:** mDNS + DNS-SD provide lightweight discovery without a central server in local networks

- **Application in project:** Implement Go-based peer discovery with automatic advertise/browse behaviour; use WLAN as primary transport.
- **Casual ordering for concurrent updates**
 - **Research Insight:** Logical clocks and event ordering reduce ambiguity when multiple peers edit concurrently.
 - **Application in project:** Track per-file version metadata (timestamps + vector-clock style state) in the coordination layer before resolving conflicts.
- **Consistency strategy under failures and partitions**
 - **Research Insight:** CAP trade-offs show that perfect consistency is difficult during partition/offline scenarios.
 - **Application in project:** Favor availability and eventual convergence on LAN; use deterministic conflict policy (LWW(Last-Write-Wins) + backup copy) and resync after reconnection.
- **Conflict-handling evolution path**
 - **Research Insight:** CRDTs can avoid some mutual conflict handling but increase design complexity.
 - **Application in project:** Keep CRDT as a future extension; current releases uses simpler LWW(Last-Write-Wins) + version-backup workflow for predictable behavior.
- **Bandwidth-efficient synchronization direction**
 - **Research Insight:** Delta-based transfer (rsync-style) reduces unnecessary data movement for large or partially changes files.
 - **Application in Project:** Start with whole-file transfer for implementation simplicity, then add block-diff optimization for larger files and multi-peer scaling.

7 Key References

1. RFC 6762 + RFC 6763 (Multicast DNS; DNS-Based Service Discovery) - foundation for LAN peer discovery and service advertisement. [RFC6762_6763]
2. Lamport, L. (1978). *Time, Clocks and the Ordering of Events in a Distributed Systems* - casual ordering baseline for concurrent file-updating reasoning. [Lamport1978]
3. Brewer, E.A. (2002). *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services* - consistency/availability trade-off baseline during node failure or disconnection. texttt[Brewer2002]
4. Shapiro, M. et al. (2011). *A comprehensive study of Convergent and Commutative Replicated Data Types* - future model for conflict-free replication in advanced versions. [Shapiro2011]
5. Tridgell, A., & Mackerras, P. (1996). *the rsync algorithm* - reference for later delta-sync optimization in large-file transfers. [Rsync1996]

8 Conclusion

- **What did you learn from the literature?**
 - Practical LAN P2P sync is best grounded in mature discovery standards (mDNS/DNS)
 - Concurrent updates require careful version tracking and deterministic conflict resolution for predictable user experience.
 - Perfect consistency is not achievable during partitions; favor availability and eventual convergence in local
 - CRDTs are promising for future conflict-free replication but add complexity; simpler LWW + backup is a good starting point.
 - Delta-based synchronization can optimize bandwidth for large files but should be added after a working whole-file transfer implementation.
- **How did the literature influence your design?**
 - We chose mDNS/DNS-SD for peer discovery to avoid central servers and simplify LAN collaboration.
 - We implemented per-file version metadata and a deterministic LWW conflict policy to handle concurrent edits predictably.
 - We designed the system to favor availability and eventual convergence during partitions, with resync after reconnection.
 - We deferred CRDT implementation to future versions, starting with a simpler versioning and backup approach for conflict handling.
 - We prioritized whole-file transfer for initial implementation, with plans to add delta-sync optimization later.
- **What are the limitations of the existing work?**
 - Many existing solutions either require central servers (Google Drive, GitHub) or lack versioning and conflict resolution (AirDrop).
 - Some systems support versioning but do not handle dynamic peer discovery or local synchronization well (Git).
 - Few solutions provide a seamless directory-level sync experience with heterogeneous file support and simple version recovery for small-scale LAN collaboration.
- **What are the next steps in your project?**
 - Implement the core peer discovery and directory synchronization features based on the insights from the literature.
 - Test the system in real-world LAN environments with multiple peers and various file types to evaluate performance and usability.
 - Plan for future enhancements, including CRDT-based conflict resolution and delta-sync optimization, based on user feedback and observed limitations.